

Sim-to-Real Transfer for Learned Legged Locomotion Policies under Contact-Rich Dynamics and Real-Time Constraints

A Literature Review

Boonyaporn Preechasuth

VISTEC Internship Examination

bpbeam.boonyaporn@gmail.com

Abstract

Deploying learned locomotion policies on physical legged robots remains a fundamental challenge due to the *sim-to-real gap*—the mismatch between simulated training environments and real-world dynamics. While deep reinforcement learning has enabled remarkable locomotion behaviors in simulation, this gap arises from inaccurate actuator modeling, simplified contact formulations, sensing noise and latency, and timing jitter that destabilize real-world control. This literature review synthesizes recent advances in sim-to-real transfer for learned legged locomotion, with focus on contact-rich dynamics and real-time constraints. I categorize the reality gap sources into four domains: actuator dynamics, contact mechanics, perception discrepancies, and temporal factors. I then review major mitigation strategies—including domain randomization, system identification, residual learning, online adaptation, and multi-simulator validation—analyzing their strengths and limitations. Finally, I discuss open challenges and propose a structured Sim-to-Sim → Sim-to-Real pipeline for robust real-world deployment.

Keywords: Sim-to-Real Transfer · Legged Locomotion · Deep Reinforcement Learning · Domain Randomization · Contact Dynamics · Online Adaptation · Quadrupedal Robots

1 Introduction

Learning-based locomotion has emerged as a powerful paradigm for controlling legged robots, representing a significant departure from traditional model-based approaches that require explicit dynamics modeling and hand-crafted gait design [18, 23]. Through deep reinforcement learning (DRL), robots can acquire agile motor behaviors—including trotting, bounding, jumping, and stair climbing—directly from interaction data, bypassing the need for manually designed motion primitives or trajectory optimization [16, 36]. The emergence of GPU-parallel physics engines has further accelerated this paradigm by enabling massively parallel simulation, where billions of environment steps can be collected within hours [26, 35].

Despite these remarkable advances, transferring learned policies from simulation to physical robots remains a fundamental and largely unsolved challenge [30, 41]. This difficulty, commonly referred to as the *sim-to-real gap* or *reality gap*, stems from the inherent mismatch between idealized simulation models and the complex, uncertain dynamics of real-world systems. Legged robots are particularly susceptible to this gap due to several distinguishing characteristics: their underactuated structure during flight phases, their fundamental reliance on intermittent and impulsive ground contacts for propulsion, and the strict real-time control requirements

imposed by dynamic stability [18]. Even minor modeling errors—whether in actuator response characteristics, contact mechanics, or sensor behavior—can destabilize the closed-loop system, resulting in falls, mechanical damage, or unsafe behaviors during physical deployment.

The practical and scientific importance of bridging this gap has motivated an extensive body of interdisciplinary research spanning robotics, machine learning, and control theory. Early approaches relied on hand-tuned domain randomization to achieve empirically robust policies [43], while contemporary methods increasingly emphasize physics-grounded modeling, online adaptation mechanisms, and systematic multi-stage validation [22, 28]. The field has rapidly progressed from simple blind locomotion on flat terrain to perceptive policies that leverage vision for terrain adaptation, and from quadrupedal platforms to increasingly complex bipedal and humanoid embodiments [34, 37].

It is important to note that the sim-to-real challenge is not unique to learning-based methods. Classical approaches—including model predictive control (MPC), trajectory optimization, whole-body control, and central pattern generators—also rely on simulation for development, tuning, and validation, and thus face similar transfer difficulties when dynamics models do not accurately reflect physical reality. However, learned policies face amplified sim-to-real challenges for several reasons: they are trained end-to-end on simulation data without explicit dynamics knowledge, they can exploit subtle simulator artifacts that classical controllers would not, and their black-box nature makes diagnosing transfer failures more difficult. This review therefore focuses on learned locomotion while acknowledging that many principles—particularly those concerning actuator modeling, contact mechanics, and latency—apply broadly across control paradigms.

Scope and Contributions: This review provides a structured analysis of the sim-to-real problem for learned legged locomotion. The contributions are threefold:

- (i) A systematic categorization of sim-to-real gap sources, organized by physical subsystem (Section 2).
- (ii) A taxonomy of transfer methodologies with critical evaluation of their merits and limitations (Section 3).
- (iii) A multi-stage Sim-to-Sim $\rightarrow$ Sim-to-Real pipeline for practical deployment (Section 5).

Paper Organization: The remainder of this paper is organized as follows. Section 2 examines the major sources of discrepancy between simulation and reality. Section 3 presents a taxonomy of transfer methodologies. Section 4 discusses current capabilities, persistent limitations, and open research challenges. Section 5 proposes a structured deployment pipeline. Section 6 concludes with perspectives on future directions.

2 Sources of the Sim-to-Real Gap

The sim-to-real gap manifests through multiple interacting mechanisms that distinguish physical reality from simulation. Following established conventions [30, 41], I organize these sources into four categories: actuator dynamics, contact mechanics, perception discrepancies, and temporal factors. Understanding each source is essential for selecting appropriate mitigation strategies.

2.1 Actuator Dynamics and Energy Dissipation

Actuator modeling represents one of the most significant yet frequently underestimated sources of the reality gap [17]. Many physics simulators treat actuators as ideal torque sources or

simplified proportional-derivative (PD) controllers with instantaneous response characteristics. In contrast, real actuators—particularly the permanent magnet synchronous motors (PMSMs) and quasi-direct-drive systems employed in contemporary quadrupeds and humanoids—exhibit complex nonlinear dynamics that profoundly influence achievable control bandwidth, torque fidelity, and energy efficiency.

2.1.1 Electrical and Mechanical Dynamics

Real electric motors are governed by coupled electrical and mechanical dynamics that introduce fundamental bandwidth limitations and energy losses. The electrical subsystem is characterized by winding inductance L , resistance R -induced power dissipation, and back-electromotive force (EMF) that opposes current flow at high velocities. The mechanical subsystem encompasses rotor inertia J , transmission friction (particularly significant in geared systems), and backlash in gear trains.

Neglecting these dynamics leads to policies that exploit unrealistic instantaneous torque availability in simulation. Such policies frequently command high-frequency, high-amplitude torque oscillations that saturate real motor drivers, exceed thermal limits, or trigger current protection mechanisms. Empirical studies report that physics-grounded actuator models are crucial for accurate energy estimation and feasible control [3, 13]. For instance, Tan et al. [41] reported that a policy trained with an ideal actuator model resulted in severe shaking and instability when deployed on the physical Minitaur robot, as the policy commanded high-frequency torques that the real motors could not track.

2.1.2 Transmission and Compliance

Geared actuators introduce additional modeling challenges through complex transmission dynamics. Gear friction varies nonlinearly across the torque-speed operating envelope, efficiency losses compound through multi-stage reduction systems, and series elastic elements (in compliant actuator designs) create coupled spring-mass dynamics. For quasi-direct-drive systems that minimize gear reduction ratios, friction modeling remains challenging because static and dynamic friction coefficients vary with temperature, lubrication state, and accumulated wear [4].

Compliance in the transmission chain or at the foot-ground interface acts as an unmodeled spring-damper system that affects contact force transmission and can excite resonant modes absent in rigid-body simulation. This phenomenon is particularly problematic for dynamic gaits where ground reaction forces must be precisely modulated for stability and efficiency.

2.2 Contact-Rich and Non-Smooth Dynamics

Legged locomotion fundamentally depends on the making and breaking of contacts with the environment, resulting in hybrid and non-smooth system dynamics characterized by discontinuous state transitions [40]. The contact problem presents both mathematical challenges (non-smoothness, complementarity constraints) and computational challenges (stiffness, convergence) that current simulators address through various approximations, each introducing characteristic artifacts.

2.2.1 Contact Formulations and Their Limitations

Rigid-body contact with Coulomb friction constitutes a complementarity problem that is notoriously difficult to simulate accurately, particularly for high-speed locomotion involving

impacts. The core challenge lies in satisfying non-penetration constraints while resolving impulsive forces instantaneously. Real-world contacts are never perfectly rigid; they involve complex deformation and friction phenomena that simple complementarity formulations fail to capture.

Simulators must approximate these physics to run in real-time, often leading to fundamental discrepancies such as soft contacts where feet sink into the ground, or hard contacts that cause numerical jitter and instability. These approximations directly affect the ground reaction forces essential for locomotion, leading to policies that may learn to exploit simulator-specific artifacts—such as generating thrust from interpenetration forces—that are impossible on physical hardware.

Each formulation makes distinct trade-offs between physical accuracy, computational efficiency, and numerical robustness. Policies trained in one simulator may inadvertently overfit to its particular contact artifacts, exhibiting qualitatively different behaviors when deployed on physical hardware or evaluated in alternative simulation environments [9, 29].

2.2.2 Friction and Deformable Substrates

Friction modeling presents additional complexity beyond the idealized Coulomb model. Real-world friction exhibits anisotropy (direction-dependence), state-dependence, and dynamic phenomena including stick-slip transitions, the Stribeck effect (velocity-dependent friction coefficient), and pressure-dependent behavior that simplified models cannot capture. Furthermore, most simulators assume rigid or quasi-rigid ground surfaces, whereas practical deployment environments include compliant substrates—grass, sand, carpet, deformable mats—that store and dissipate energy during stance phases through viscoelastic deformation [14].

The interaction between foot geometry, terrain material properties, and gait dynamics creates a rich combinatorial space of contact behaviors. Policies trained exclusively on rigid terrain may generate foot trajectories that fail catastrophically on soft substrates (excessive sinking, insufficient toe clearance) or slippery surfaces (inadequate friction margin for commanded forces).

2.3 Perception and Sensing Discrepancies

Simulation environments typically provide access to noise-free, ground-truth state information sampled at precisely specified intervals with perfect synchronization. Real robotic perception operates under fundamentally different conditions characterized by noise, bias, latency, and limited observability—introducing discrepancies that can cause catastrophic control failures if not adequately addressed during training [28].

2.3.1 Proprioceptive Sensing

Proprioceptive measurements from joint encoders, motor current sensors, and inertial measurement units (IMUs) suffer from multiple noise sources with distinct statistical characteristics:

Encoder Quantization: Finite encoder resolution limits joint position accuracy, with effects particularly pronounced at low velocities where quantization noise dominates velocity estimates obtained through numerical differentiation. Multi-turn absolute encoders mitigate wrap-around issues but introduce additional complexity.

IMU Characteristics: Gyroscope measurements exhibit bias drift (random walk), accelerometer signals contain high-frequency noise and are corrupted by vibration, and cross-axis sensitivity introduces coupling between measurement channels. Attitude estimation filters (complementary, extended Kalman, or complementary) impose dynamics that create phase lag and may exhibit divergent behavior during high-acceleration maneuvers or prolonged motion.

Delay and Filtering: Analog anti-aliasing filters, digital decimation, communication latency between distributed sensors and central controllers, and state estimation filter dynamics all introduce delays between physical events and their observation by the control policy.

These effects become especially critical during underactuated flight phases in running gaits or bipedal locomotion, where accurate state estimation is essential for proper touchdown timing and pre-impact leg positioning [24, 37].

2.3.2 Exteroceptive Sensing

Vision-based locomotion policies face an even larger sim-to-real gap. Synthetically rendered images differ systematically from real camera observations across multiple dimensions:

Low-Level Statistics: Pixel noise characteristics, motion blur during fast locomotion, rolling shutter distortion, and automatic exposure/gain variations all differ between simulated and real imagery.

Appearance: Material reflectance properties (BRDF), complex lighting conditions with multiple light sources and shadows, and accurate color reproduction are computationally expensive to simulate with photorealistic fidelity.

Geometric Artifacts: Lens distortion, depth sensor noise patterns (particularly for structured-light or time-of-flight sensors), and interference between multiple active sensors create measurement artifacts absent in idealized simulation.

Without explicit domain adaptation, careful visual domain randomization, or learned domain-invariant representations, purely vision-driven policies may rely on spurious features present only in simulation, causing complete perception failure upon deployment [1, 45].

2.4 Temporal Constraints and Latency

Real robotic systems operate under strict real-time constraints and are inherently asynchronous, with different subsystems (sensing, computation, actuation) running at different rates and with variable phase relationships. Delays accumulate throughout the control pipeline, and timing jitter—stochastic variation in loop execution time—can destabilize feedback control even when average latency remains small [41].

2.4.1 Sources of Latency

The total round-trip delay from physical event to motor response comprises contributions from multiple pipeline stages:

- (1) **Sensor Acquisition:** Physical transduction time, analog-to-digital conversion, sensor-local signal conditioning and filtering.
- (2) **Communication:** Bus protocol overhead (CAN, EtherCAT, USB), message serialization, and network topology-dependent propagation delays.

- (3) **Computation:** Neural network forward pass inference time, state estimation filter updates, and action post-processing (smoothing, saturation, safety checks).
- (4) **Actuation:** Motor driver PWM update latency, current controller bandwidth limitations, and electromechanical time constants of the motor-load system.

For neural network policies, inference latency depends jointly on network architecture complexity and deployment hardware. While modern GPU accelerators can evaluate large transformer or convolutional networks in sub-millisecond time, embedded CPU or microcontroller deployment may require tens of milliseconds—approaching or exceeding typical control periods of 1–10 ms required for dynamic locomotion stability [35].

2.4.2 Timing Jitter and Asynchrony

Beyond deterministic mean latency, stochastic variability in loop timing disrupts the fixed-timestep assumptions implicit in standard simulation. Jitter sources include:

- Operating system task scheduling in non-real-time environments
- Garbage collection pauses in managed runtime languages (Python, Java)
- Data-dependent neural network execution time (e.g., early-exit networks, dynamic computation graphs)
- Communication bus contention and retry mechanisms
- Thermal throttling of processors under sustained computational load

Policies trained with perfectly synchronized, deterministic sensing and actuation may exhibit oscillations, phase lag instability, or limit cycle behaviors when deployed on systems with non-negligible latency variance. This effect is particularly pronounced for high-bandwidth force or impedance control loops where phase margin provides critical stability assurance.

3 Taxonomy of Sim-to-Real Transfer Methods

The robotics and machine learning communities have developed diverse approaches to bridge the sim-to-real gap, ranging from purely data-driven randomization to physics-informed hybrid architectures. These methods address discrepancies at different levels: the environment model, the policy architecture, or the data distribution. I organize these methods into seven major categories (Table 1), emphasizing that state-of-the-art systems typically employ a multi-layered approach combining several of these techniques.

Table 1: Comprehensive Taxonomy of Sim-to-Real Transfer Methods for Legged Locomotion

Method Category	Core Principle	Representative Works
Domain Randomization	Train over parameter distributions $P(\nu)$ containing reality	[27, 32, 43]
System Identification	Optimize $\min_z \ s_{real} - s_{sim}(z)\ $ to align parameters	[7, 17, 46]
Residual Learning	Learn corrections Δa to a nominal controller π_{base}	[19, 21, 38]
Online Adaptation	Infer environment context z from proprioceptive history	[20, 22, 28]
Domain Adaptation	Align feature distributions $P(f s_{sim})$ and $P(f s_{real})$	[15, 33]
Meta-Learning	Learn to adapt to new tasks with few-shot samples	[11, 12]
Imitation Learning	Match expert trajectories τ_{real} from physical hardware	[6, 31]
Hybrid Control	Integrate RL with Model Predictive Control (MPC)	[14, 25]
Sim-to-Sim Validation	Use multi-simulator consensus as a transfer proxy	[9, 29]
Physics Engine Selection	Leverage appropriate contact and integration solvers	[2, 44]

3.1 Domain Randomization

Domain randomization (DR) enhances policy robustness by training over distributions of simulated environments with randomized physical parameters [32, 43]. The fundamental hypothesis underlying DR is that if a policy is sufficiently robust to perform well across a broad distribution of simulated environments, the real-world environment will effectively manifest as another sample within that distribution.

3.1.1 Mathematical Formulation

Formally, we consider a family of Markov Decision Processes (MDPs) $\mathcal{M}(\nu)$, where $\nu \sim P(\nu)$ represents a vector of physical parameters (e.g., mass, friction, latency) sampled from a distribution P . The goal is to find a policy π_θ that maximizes the expected return across the entire distribution:

$$J(\theta) = \mathbb{E}_{\nu \sim P(\nu)} \left[\mathbb{E}_{\tau \sim p(\tau|\theta, \nu)} \left[\sum_{t=0}^T \gamma^t r(s_t, a_t) \right] \right] \quad (1)$$

where $\tau = (s_0, a_0, s_1, a_1, \dots)$ is a trajectory. By exposing the agent to a wide range of dynamics during training, the network is forced to learn features and control strategies that are invariant to the specific values of ν , or that implicitly estimate ν from the observation history.

3.1.2 Implementation Scope and Practice

Implementing domain randomization requires modifying the simulation loop to sample and apply physical parameters at the start of each episode or at fixed intervals. In modern GPU-accelerated simulators like Isaac Gym or Isaac Lab [26], this is efficiently handled by vectorizing the environment instances, where each instance i operates with a unique parameter set ν_i .

Implementation Architecture: The randomization process typically involves three steps:

1. **Parameter Sampling:** Before an episode begins, parameters ν are sampled from their respective distributions (e.g., $m \sim \mathcal{U}[m_{min}, m_{max}]$).
2. **Physics Application:** The sampled parameters are applied to the rigid body and joint properties in the physics engine. For example, modifying the link inertia tensor $\mathbf{I}$ or the joint friction coefficient μ .
3. **Observation Noise Injection:** During execution, sensor readings are corrupted by additive noise $o_t = o_t^{clean} + \epsilon$, where ϵ is sampled at every timestep to simulate sensor noise.

Standard domain randomization protocols randomize parameters across several categories:

Dynamics Parameters: Body segment masses $m_i \sim \mathcal{U}[m_i^{min}, m_i^{max}]$, link inertia tensors, payload masses and their mounting locations.

Surface Properties: Ground friction coefficients $\mu \sim \mathcal{U}[\mu^{min}, \mu^{max}]$, terrain stiffness, coefficient of restitution for impact modeling.

Actuator Characteristics: Motor torque constants, PD gains, joint damping, torque limits, and control delay magnitudes.

Sensor Noise: Additive Gaussian noise on observations $o_t \leftarrow o_t + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma^2)$, multiplicative scale variations, bias terms, and observation dropout with probability p_{drop} .

External Perturbations: Random push forces applied to the robot base, terrain height variations, wind disturbances.

The randomization schedule admits several design choices. *Episode-level randomization* samples parameters at episode initialization and holds them constant throughout, exposing the policy to diverse but stationary environments. *Continuous randomization* varies parameters during rollouts according to temporal dynamics.

Locomotion Successes:

- **Minitaur (Tan et al. [41]):** Successfully transferred a trotting gait to the Minitaur robot by randomizing link masses, friction, and motor damping. This work was among the first to demonstrate that a policy learned purely in simulation could work on a physical quadruped without fine-tuning.

Failure Modes:

- **Conservative Gaits:** Excessive randomization often leads to shuffling gaits where the robot keeps feet close to the ground/center-of-mass to maximize stability, sacrificing speed and energy efficiency [30].
- **Reality Gap in Unmodeled Dynamics:** If the randomization distribution does not cover the true physical parameters (e.g., unexpected backlash), the policy may fail catastrophically [43].

Conclusion: Standard domain randomization is a powerful, assumption-free starting point but requires careful tuning to avoid overly conservative behaviors.

3.1.3 Automatic and Adaptive Randomization

Manual specification of appropriate randomization ranges is labor-intensive, requires domain expertise, and may inadvertently produce distributions that are either overly broad (yielding conservative policies) or overly narrow (excluding legitimate real-world conditions). Several principled approaches automate this specification:

Active Domain Randomization (ADR): Progressively expands randomization ranges as policy competence improves, maintaining the randomization distribution at the frontier of policy capability [27]. This curriculum-based approach ensures consistent training difficulty throughout learning, analogous to the terrain curricula used in blind locomotion training [23].

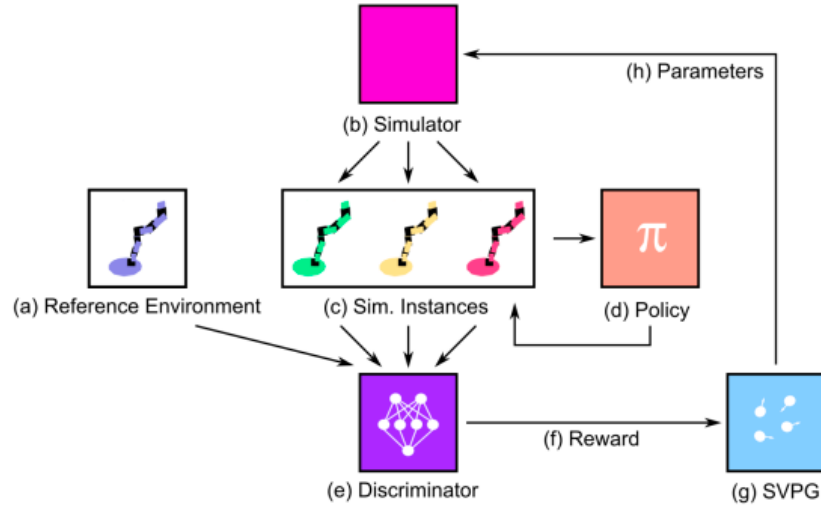


Figure 1: ADR continuously updates environment complexity by using a discriminator to distinguish between randomized and reference rollouts, guiding a particle-based generator to propose challenging parameters for the agent.

Curriculum Learning: Structures the randomization distribution according to estimated policy competence, beginning with narrow ranges around nominal parameters and systematically expanding to more challenging variations as performance thresholds are satisfied.

Mechanism Details: The core of DR lies in the distribution choice. Simple *Uniform Domain Randomization* (UDR) draws parameters from fixed intervals (e.g., mass $\pm 10\%$). More advanced *Automatic Domain Randomization* (ADR) adjusts these ranges based on policy performance, creating a curriculum that prevents the policy from learning conservative safety behaviors early on. The goal is to maximize the entropy of the training distribution while maintaining physical feasibility.

Locomotion Successes:

- **ANYmal (Lee et al.):** Trained on procedurally generated terrain with randomized friction and height maps. The policy demonstrated zero-shot transfer to mud, snow, and stairs, proving that randomizing terrain geometry is as crucial as randomizing dynamics.



Figure 2: ANYmal training on procedurally generated terrain with randomized height maps and friction. Randomizing geometry alongside dynamics enables robust proprioceptive policies that transfer zero-shot to real-world obstacles. [Image: Lee et al. 2020]

Failure Modes:

- **Conservative Gaits:** As with standard DR, excessive randomization range expansion can lead to sub-optimal, risk-averse behaviors that prioritize stability over performance [30].
- **Jitter and Oscillation:** If observation noise is randomized too aggressively (e.g., > 0.1 rad noise on joints), policies may develop high-frequency twitching to maximize signal-to-noise ratio or exploit simulator integration errors [41].

Conclusion: Active approaches like ADR automate the difficult tuning process of DR, pushing the boundaries of policy robustness, though they still fundamentally trade off optimality for robustness.

3.2 System Identification

System identification (SysID) directly addresses the sim-to-real gap by optimizing simulation parameters to minimize discrepancy with real-world observations [7, 46]. In contrast to domain randomization's implicit, distribution-based approach, SysID explicitly estimates physical parameters from measurement data.

3.2.1 Classical Parametric Approaches

Traditional system identification applies established techniques from control engineering to individual robot subsystems:

Actuator Identification: Fitting motor electrical parameters (winding resistance R , inductance L , torque constant k_τ), friction characteristics (Coulomb, viscous, Stribeck terms), and bandwidth limitations from controlled current-voltage-velocity measurements under various loading conditions.

Link Dynamics: Estimating link masses m_i , centers of mass $\mathbf{r}_{\text{com},i}$, and inertia tensors $\mathbf{I}_i$ using regression on the inverse dynamics equation $\boldsymbol{\tau} = \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q})$ with known trajectories.

Contact Parameters: Inferring effective friction coefficients μ and contact stiffness k_c from force-velocity relationships during controlled sliding and impact experiments.

These approaches require careful experimental protocols and appropriate instrumentation but yield physically interpretable parameters with potential for generalization beyond the identification dataset.

Locomotion Successes:

- **MIT Cheetah 3 (Bledt et al.):** Classical identification of link inertia and motor parameters enabled high-performance model-based control (MPC), which serves as a strong baseline for or component of learned systems [46].

Failure Modes:

- **Parameter Coupling:** Identifiable parameters are often coupled (e.g., friction and motor damping), leading to multiple local minima where individual parameters are wrong but their sum explains the data, causing failure when conditions change [30].
- **Non-Stationarity:** Fixed parametric models cannot account for time-varying effects like temperature-induced friction changes.

Conclusion: Classical SysID provides the grounded physics model necessary for simulation but struggles with the complex, non-linear, and time-varying nature of modern soft/compliant actuators.

3.2.2 Learned Actuator Models: ActuatorNet

Classical analytical actuator models often fail to capture the full complexity of real motor dynamics, particularly nonlinear effects such as friction hysteresis, thermal derating, and velocity-dependent efficiency losses. Hwangbo et al. [17] proposed ActuatorNet, a data-driven approach that learns the actuator dynamics directly from hardware measurements using neural networks.

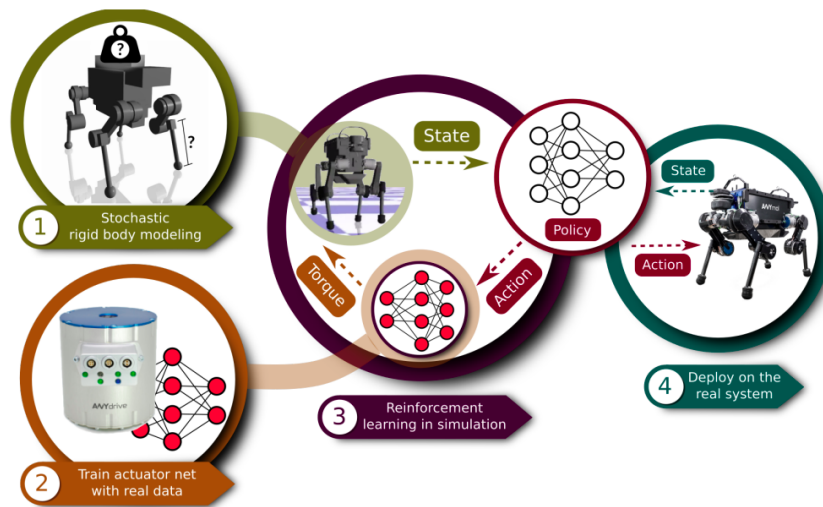


Figure 3: ActuatorNet architecture. The network takes a history of joint states and commands as input and predicts the actual torque produced by the hardware, capturing complex nonlinearities like Back-EMF and friction that are difficult to model analytically.

Mechanism Details: ActuatorNet relies on high-fidelity telemetry data collected from the physical robot. The network $\mathcal{A}_\phi$ is trained to map a history of motor states to the realized torque.

- **Data Collection:** Real-world data pairs $(\tau_{\text{cmd}}, \mathbf{q}, \dot{\mathbf{q}}) \rightarrow \tau_{\text{real}}$ are collected by driving the robot through chirp signals (sweeping frequencies from 0.1Hz to 20Hz) or random motor babbling trajectories to cover the entire state-space of the actuator.
- **Network Architecture:** The network is typically structured as a causal Temporal Convolutional Network (TCN) or a recurrent layer (e.g., GRU) with a history window of H steps (e.g., $H = 20$ ms). The input vector at time t usually consists of the position error $q_{\text{des}} - q_t$, velocity $\dot{q}_t$, and previous commanded torques.
- **Inference:** During simulation, the physics engine's ideal torque source is replaced by the forward pass of ActuatorNet: $\tau_{\text{sim}} = \mathcal{A}_\phi(H_t)$.

By minimizing the discrepancy:

$$\mathcal{L}_{\text{net}} = \sum_k \|\tau_{\text{real}}(k) - \text{NN}(\mathbf{a}_{t-n:t}, \mathbf{q}_{t-n:t}, \dot{\mathbf{q}}_{t-n:t})\|^2 \quad (2)$$

the simulation can accurately emulate soft constraints, back-EMF drop-off, and bandwidth limits of real actuators. This learned actuator model then replaces the ideal motor model in the physics engine, preventing the policy from commanding high-frequency signals that the real hardware cannot track.

Locomotion Successes:

- **ANYmal (Hwangbo et al.):** By replacing the ideal DC motor model with ActuatorNet, the simulation accurately predicted the torque drop-off at high speeds due to back-EMF. This allowed the policy to learn efficient running gaits that respected the true physical limits of the hardware, achieving record-breaking speed and energy efficiency on the ANYmal B robot.

Failure Modes:

- **Overfitting to Unit-Specifics:** A model identified for Robot A might not work for Robot B of the same series due to manufacturing variations, requiring re-identification for every deployed unit [30].
- **Hardware Drift:** As gearboxes wear down and lubrication properties change with temperature, the identified model becomes stale. Using an ActuatorNet trained in summer might fail in winter outdoor deployment without continuous re-identification [22].

Conclusion: Learned actuator models provide the highest fidelity simulation and are critical for pushing hardware to its limits (e.g., racing). However, they are brittle and expensive to maintain compared to robust control methods.

3.2.3 Differentiable Simulation and Gradient-Based Optimization

Recent advances in differentiable physics simulation enable gradient-based optimization of simulation parameters by backpropagating through the dynamics. Given a trajectory matching objective:

$$\mathcal{L}(z) = \sum_{t=0}^T \left\| \mathbf{s}_t^{\text{real}} - \mathbf{s}_t^{\text{sim}}(z) \right\|_{\mathbf{W}}^2 \quad (3)$$

where $z \in \mathbb{R}^d$ denotes the vector of simulation parameters, $\mathbf{s}_t$ represents states at time t , and $\mathbf{W}$ is a positive-definite weighting matrix, the gradients $\nabla_z \mathcal{L}$ can be computed through automatic

differentiation of the simulation dynamics, enabling efficient first-order optimization.

Differentiable simulators have demonstrated state prediction errors reduced by one to two orders of magnitude compared to nominal parameter values. Given a differentiable dynamics model $\mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{a}_t, \mathbf{z})$, we can compute the sensitivity of the loss with respect to simulation parameters using the chain rule:

$$\frac{d\mathcal{L}}{dz} = \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial \mathbf{s}_t} \left(\frac{\partial \mathbf{s}_t}{\partial z} + \frac{\partial \mathbf{s}_t}{\partial \mathbf{s}_{t-1}} \frac{\partial \mathbf{s}_{t-1}}{\partial z} \right) \quad (4)$$

This allows for rapid convergence to the parameters that best explain real-world trajectories. However, practical deployment requires differentiable contact handling (typically achieved through smooth approximations to the complementarity conditions) and careful initialization to avoid convergence to poor local minima in the non-convex parameter landscape.

Locomotion Successes:

- **System Identification (Chebotar et al.):** Differentiable optimization of simulation parameters (ground friction, actuator gains) allowed the policy to close the reality gap for a quadruped robot, significantly outperforming random search.

Failure Modes:

- **Local Minima:** Gradient-based optimization is prone to getting stuck in poor local minima, especially given the highly non-convex loss landscapes of contact-rich locomotion tasks.
- **Contact Smoothing Artifacts:** To make contact differentiable, solvers often use soft approximations that can lead to unphysical interpenetration or sticky contact behaviors.

Conclusion: Differentiable simulation offers a mathematically elegant path to system identification but is currently limited by the non-convexity of contact mechanics and computational scale.

3.3 Residual Reinforcement Learning

Residual learning architectures decompose the control policy into a fixed baseline—typically derived from classical control theory or trajectory optimization—and a learned residual that compensates for unmodeled effects [38]. This hybrid structure provides baseline stability guarantees while retaining the flexibility of learned adaptation.

3.3.1 Architectural Formulation

A residual policy decomposes the action as a sum of a base controller contribution and a learned residual:

$$\mathbf{a}_t = \mathbf{a}_t^{\text{base}} + \Delta \mathbf{a}_t^{\text{learned}} = \pi_{\text{base}}(\mathbf{s}_t) + \pi_{\theta}(\mathbf{s}_t) \quad (5)$$

Mathematical Objective: From the perspective of RL, the baseline controller π_{base} modifies the MDP transition dynamics perceived by the learner. The residual policy is optimized to maximize the standard discounted return:

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{\pi_{\text{base}} + \pi_{\theta}} \left[\sum_{t=0}^{\infty} \gamma^t r(\mathbf{s}_t, \pi_{\text{base}}(\mathbf{s}_t) + \pi_{\theta}(\mathbf{s}_t)) \right] \quad (6)$$

By defining the action space relative to π_{base} , we essentially warm-start the policy in a high-reward region of the state space, as the robot is already walking or standing.

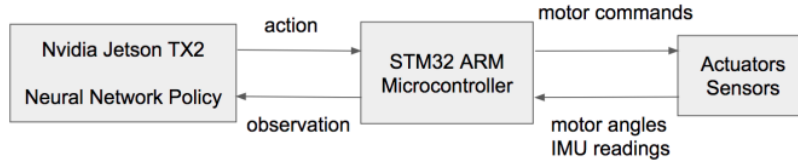


Figure 4: Residual Reinforcement Learning Architecture. The final action a is the sum of a nominal policy π_{base} (e.g., CPG or PID) and a learned residual π_{res} . This structure ensures baseline stability while allowing learning to handle unmodeled dynamics.

where π_{base} represents a fixed or slowly-adapted baseline controller (e.g., model-predictive control, PD tracking of reference trajectories, central pattern generators), π_{residual} (parameterized by θ) denotes the learned neural network correction. In practice, a mixing coefficient α is often used to bound the residual: $\mathbf{a}_t = \pi_{\text{base}} + \alpha \tanh(\text{NN}(\mathbf{s}_t))$.

3.3.2 Advantages and Applications

This architectural decomposition offers several compelling advantages:

Safety Prior: The baseline controller ensures reasonable nominal behavior even when the learned residual produces unexpected or out-of-distribution outputs, providing an implicit safety envelope.

Sample Efficiency: Learning small corrections to a competent baseline requires less exploration and fewer samples than learning complete policies from scratch, particularly for high-dimensional action spaces.

Interpretability: The baseline contribution provides physical intuition about the policy’s intent, while the residual captures unmodeled corrections, facilitating debugging and analysis.

Regularization: Constraining the residual magnitude (e.g., through action penalty terms) naturally regularizes the learned component, improving generalization.

Mechanism Details: The architecture explicitly separates duties: π_{base} handles the nominal task (e.g., keeping the robot upright and moving forward), while π_{residual} handles the perturbations (e.g., correcting for a slip or push). This formulation transforms the RL problem from learning to walk to learning to correct walking, which has a much easier exploration landscape.

Locomotion Successes:

- **Laikago (Iscen et al.):** Used a simple sine-wave generator as the baseline. The residual policy learned to modulate the phase and amplitude of these waves. The result was a robot that could trot robustly over slopes and uneven terrain, significantly outperforming the open-loop baseline.

Failure Modes:

- **Policy Conflict:** If the baseline is fundamentally unsuited for the terrain (e.g., a trotting generator on terrain requiring a static crawl), the residual term π_{residual} must expend its entire output range “canceling out” the baseline before it can apply the correct action. This creates an internal conflict that leads to energy waste and poor control authority [38].

Conclusion: Residual learning is the safest bridge for deploying learning on expensive hardware. It guarantees a minimum level of performance but is ultimately limited by the quality and flexibility of the baseline controller.

3.4 Online Adaptation with Latent Representations

Online adaptation frameworks equip robots with the capability to infer hidden environmental properties and dynamically adjust behavior during deployment, without requiring explicit system identification or retraining [20, 22]. These methods embed environment-specific information in learned latent representations that modulate policy behavior.

3.4.1 Teacher-Student Architectures: Rapid Motor Adaptation

The Rapid Motor Adaptation (RMA) framework [22] employs a two-phase teacher-student training procedure:

Phase 1: Teacher Training. A teacher policy $\pi_{\text{teacher}}(\mathbf{a}|\mathbf{o}_t, \mathbf{e}_t)$ is trained via Reinforcement Learning (typically PPO) to maximize the expected return:

$$\mathcal{L}_{\text{RL}} = \mathbb{E}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right] \quad (7)$$

where the teacher has privileged access to the extrinsic vector $\mathbf{e}_t \in \mathbb{R}^k$ encoding ground-truth environment parameters (e.g., friction, mass, slope) that are typically unavailable on hardware.

Phase 2: Adaptation Module Training. An adaptation module (or student) $\phi : (\mathbf{o}_{t-H:t}) \mapsto \hat{\mathbf{z}}_t$ is trained to predict the latent representation $\mathbf{z}_t = \mu(\mathbf{e}_t)$ produced by the teacher's extrinsic encoder. The training objective is typically a supervised reconstruction loss:

$$\mathcal{L}_{\text{adaptation}} = \mathbb{E} \|\text{StopGradient}[\mu(\mathbf{e}_t)] - \phi(\mathbf{o}_{t-H:t})\|^2 \quad (8)$$

where $\mathbf{o}_{t-H:t}$ is a history of H proprioceptive observations. This allows the robot to infer the physical context solely from the temporal signature of its own movements.

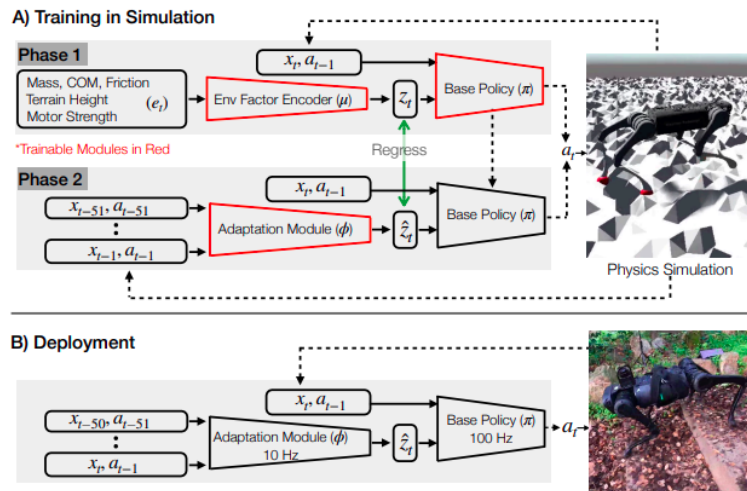


Figure 5: RMA Framework: (Phase 1) A teacher policy learns with privileged access to physical parameters. (Phase 2) A student policy adapts online by estimating these parameters from proprioceptive history via an adaptation module.

Mechanism Details: The key to RMA is the *asymmetric information flow*. The implementation involves two distinct neural networks trained in stages:

1. **Teacher Policy ($\pi_{teacher}$):** A standard MLP trained via PPO. It receives the state s_t and the ground-truth environment vector e_t (containing friction, mass, terrain height).
2. **Adaptation Module (μ_ϕ):** A 1D Convolutional Neural Network (CNN) or TCN that takes a history of proprioceptive observations (e.g., $o_{t-50:t}$) as input. It outputs a low-dimensional extrinsics vector estimate $\hat{z}_t$.
3. **Student Policy ($\pi_{student}$):** A copy of the teacher’s structure, but it takes $\hat{z}_t$ (from the adaptation module) instead of the ground-truth e_t .

During deployment, only the Adaptation Module and Student Policy structure are used. The Adaptation Module acts as a virtual sensor, inferring physical properties like friction from the history of joint discrepancies. For example, if the robot’s feet are slipping, the history of joint velocities will differ from the commanded velocities, and the module infers a low friction latent code.

Locomotion Successes:

- **Unitree A1 (Kumar et al. [22]):** Demonstrated traversing an oil-slicked plastic sheet. The adaptation module detected the slip within fractions of a second and the policy switched to a high-frequency, low-stride gait.
- **Detecting Payloads:** The same network could estimate when a 12kg payload was thrown onto the robot’s back, instantly increasing torque gains to maintain body height.

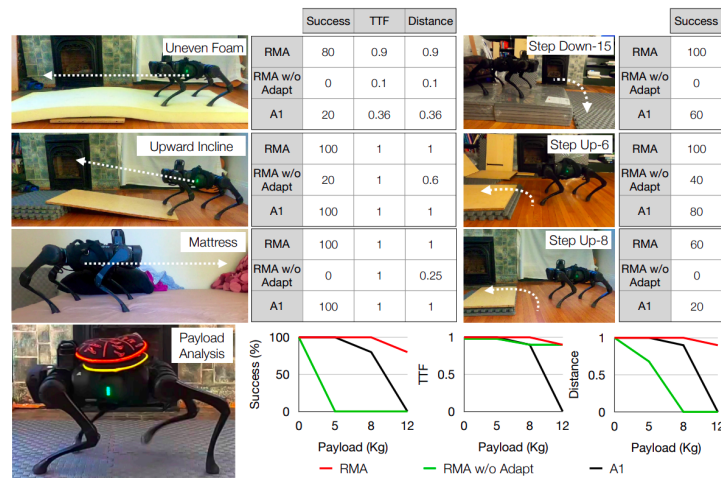


Figure 6: Real-world deployment of RMA on a Unitree A1 robot. The adaptation module enables robust locomotion over challenging terrains and under varying load conditions by estimating extrinsic parameters from proprioception.

Failure Modes:

- **Out-of-Distribution Adaptation:** If the robot encounters a perturbation not seen during the teacher’s training (e.g., getting entangled in vines), the adaptation module maps this unknown signature to an arbitrary latent vector. This can cause the policy to emit nonsensical actions, leading to falls [22].
- **Adaptation Lag:** The module needs a history window (e.g., 0.5s) to infer parameters. For sudden, impulse-like disturbances (like a mid-air collision), the adaptation is often too slow to react [20].

Conclusion: Online adaptation is the current state-of-the-art for deploying robust policies. It combines the optimality of knowing the physics with the practicality of blind deployment. Limitations remain in the speed of adaptation and generalization to completely novel physics phenomena.

3.4.2 Concurrent Training Approaches

Ji et al. [20] proposed concurrent joint optimization of the adaptation module and base policy, avoiding the sequential staged training of teacher-student methods. By simultaneously training π and ϕ with shared gradients, the system learns representations $\hat{e}$ that are specifically useful for control decisions rather than accurate environment reconstruction per se. This end-to-end approach can discover task-relevant latent structures that may differ from pre-specified extrinsic parameterizations.

Locomotion Successes:

- **Slippery Terrain (Ji et al.):** Demonstrated robust high-speed running on low-friction surfaces by concurrently learning to estimate contact probability and friction, allowing the robot to modulate its gait cycle in real-time.

Failure Modes:

- **Training Instability:** Co-training the estimator and policy can lead to learning instabilities or latent collapse, where the estimator fails to learn meaningful features because the policy finds a local optimum that ignores the estimator's input [8].

Conclusion: Concurrent training simplifies the pipeline by removing the need for a separate teacher policy but requires careful tuning to ensure both components learn effectively.

3.4.3 Implicit Adaptation through Recurrence

Recurrent neural network architectures (LSTM, GRU, Transformers) naturally maintain temporal context and can implicitly adapt to changing conditions through hidden state dynamics, without explicit environment inference modules. While less interpretable than explicit latent-variable adaptation, recurrent policies can capture complex temporal dependencies and long-horizon correlations, achieving competitive performance on challenging locomotion benchmarks [28]. The hidden state serves as an implicit belief over environment parameters, updated through observations.

Locomotion Successes:

- **Wild Locomotion (Miki et al.):** A recurrent belief-state encoder enabled the ANYmal robot to navigate extreme terrain (snow, vegetation) by fusing proprioception and noisy exteroception, implicitly adapting to sensor degradation.

Failure Modes:

- **Memory Horizon Limits:** Recurrent networks have a finite effective memory horizon. If environmental changes occur over timescales longer than this horizon (e.g., slow thermal drift), the policy may fail to track the change [30].

Conclusion: Implicit adaptation is a powerful, black-box approach that excels in complex, unstructured environments where explicit parameter identification is difficult by definition.

3.5 Domain Adaptation and Feature Alignment

Domain adaptation methods aim to learn state representations that are invariant to the shift between simulation and reality. Instead of randomizing the environment, these methods explicitly align the distributions of simulated and real data in a latent feature space.

3.5.1 Adversarial Feature Alignment

Inspired by Domain-Adversarial Neural Networks (DANN), this approach uses a domain discriminator D that tries to distinguish whether a feature $f = E(s)$ originated from simulation or reality. The encoder E is trained to maximize the discriminator's loss, effectively "fooling" it and producing domain-invariant features:

$$\min_{E, \pi} \max_D \mathcal{L}_{\text{task}}(E, \pi) - \lambda \mathcal{L}_{\text{domain}}(E, D) \quad (9)$$

In legged locomotion, this prevents the policy from overfitting to simulation-specific textures or sensor noise patterns that do not exist on hardware.

3.5.2 Sim-to-Real Image Translation

For vision-based policies, generative models like CycleGAN or UNIT can be used to translate simulated images into the real-world domain before passing them to the policy. By training on unpaired sim and real images, the model learns to add realistic lighting, noise, and motion blur to synthetic data, narrowing the visual reality gap.

3.6 Meta-Learning and Few-Shot Adaptation

Meta-learning, or learning to learn, focuses on training policies that can rapidly adapt to new environments using only a few interaction samples from the real world.

3.6.1 Model-Agnostic Meta-Learning (MAML)

MAML optimizes for a set of initial weights θ such that a small number of gradient steps on a new task $\mathcal{T}_i$ lead to high performance. The meta-objective is:

$$\min_{\theta} \sum_{\mathcal{T}_i} \mathcal{L}_{\mathcal{T}_i}(\theta - \alpha \nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(\theta)) \quad (10)$$

For a legged robot, tasks might correspond to different floor materials or slope angles. During deployment, the robot performs a "rollout" to collect data, computes a gradient, and updates its policy in real-time to match the current terrain.

3.6.2 Context-Conditional Meta-Learning

Recent works use Variational Information Bottleneck (VIB) or contrastive learning to extract a context vector c from a few real-world steps. The policy $\pi(a|s, c)$ is then conditioned on this context, allowing for instantaneous (zero-shot or few-shot) adaptation without explicit gradient updates during deployment. This bridges the gap by treating the real world as a "new task" within a learned multi-task space.

3.7 Imitation Learning and Expert Trajectory Alignment

Imitation learning (IL) leverages high-quality demonstrations—often from motion capture, human masters, or high-fidelity trajectory optimizers—to guide the learning process toward physically viable and aesthetically natural gaits. This minimizes the risk of learning "shuffling" or "jittery" gaits that are common in pure RL.

3.7.1 Behavioral Cloning (BC)

The simplest form of IL is supervised learning on state-action pairs (s, a^*) from an expert π_E . The goal is to minimize the MSE:

$$\mathcal{L}_{BC}(\theta) = \mathbb{E}_{(s, a^*) \sim \rho_E} [\|\pi_\theta(s) - a^*\|^2] \quad (11)$$

While efficient, BC suffers from "distributional shift" when the agent enters states not covered by the expert, often leading to rapid error accumulation and failure.

3.7.2 Generative Adversarial Imitation Learning (GAIL)

GAIL treats the transfer problem as a distribution matching task. A discriminator D_ψ is trained to distinguish between expert trajectories and agent trajectories, while the policy π_θ is trained to deceive the discriminator. The objective is:

$$\min_{\theta} \max_{\psi} \mathbb{E}_{\pi_\theta} [\log D_\psi(s, a)] + \mathbb{E}_{\pi_E} [\log(1 - D_\psi(s, a))] - \lambda H(\pi_\theta) \quad (12)$$

By matching the first and second-order statistics of expert motion, GAIL produces highly realistic locomotion that often transfers better to reality because the learned patterns respect the implicit regularities (e.g., center-of-mass swings) of natural movement.

3.8 Hybrid Control: Integrating RL with MPC

Hybrid architectures combine the long-term planning and robust stabilization of Model Predictive Control (MPC) with the high-bandwidth, non-linear adaptation of Reinforcement Learning.

3.8.1 RL as an MPC Parameter Tuner

In this configuration, the RL policy does not output torques or positions directly. Instead, it outputs high-level parameters for the MPC's cost function or dynamics model (e.g., desired foot placement offsets, body height, or contact schedules). The MPC then solves the following optimization at each timestep k :

$$\begin{aligned} \min_{\mathbf{u}_{k:k+N}} \quad & \sum_{i=k}^{k+N} \|\mathbf{x}_i - \mathbf{x}_{\text{ref}}(\hat{\theta})\|_{\mathbf{Q}}^2 + \|\mathbf{u}_i\|_{\mathbf{R}}^2 \\ \text{s.t.} \quad & \mathbf{x}_{i+1} = \mathbf{A}(\hat{\theta})\mathbf{x}_i + \mathbf{B}(\hat{\theta})\mathbf{u}_i + \mathbf{d} \end{aligned} \quad (13)$$

where $\hat{\theta} = \pi_{\text{RL}}(s_k)$ are the RL-tuned parameters. This leverages the physical consistency of MPC (e.g., hard constraints on ground reaction forces) while allowing RL to compensate for terrain-induced modeling errors.

3.8.2 Differentiable MPC-in-the-loop

Recent breakthroughs allow backpropagating through the MPC solver itself. This enables end-to-end training of the RL-MPC hierarchy, where the RL policy learns to pre-compensate for MPC inaccuracies by anticipating future state deviations. This combination is particularly effective for high-speed running on complex terrain.

3.9 Sim-to-Sim Validation

Sim-to-sim validation evaluates learned policies across multiple physics simulators with different numerical implementations, contact models, and integration schemes, serving as a predictive proxy for real-world transfer before hardware deployment [9, 29].

3.9.1 Rationale and Methodology

The guiding principle is that policies exhibiting consistent performance across simulators with diverse numerical characteristics likely rely on robust, physically-grounded behaviors rather than simulator-specific computational artifacts. Conversely, policies whose performance degrades sharply across simulators are probably exploiting idiosyncratic features of the training environment that will not persist in reality.

Mathematical Consistency Metric: We can quantify policy transferability by evaluating the policy π across N different simulation environments $\mathcal{S}_1, \dots, \mathcal{S}_N$. A consistency score $C(\pi)$ can be defined as the inverse of the performance variance:

$$C(\pi) = \left(\frac{1}{N-1} \sum_{i=1}^N (R_i(\pi) - \bar{R}(\pi))^2 \right)^{-1} \quad (14)$$

where $R_i(\pi)$ is the average return in simulator $\mathcal{S}_i$. A high $C(\pi)$ indicates that the policy has learned a "consensus" physical strategy that is invariant to the choice of integrator or contact solver.

Common cross-validation platforms include:

MuJoCo [44]. Constraint-based contact formulation via projected Gauss-Seidel solvers; implicit Euler integration.

Isaac Gym / Isaac Sim [26]. GPU-parallel simulation with penetration-based contact models; supports massive parallelism for rapid training.

PyBullet [10]. Sequential impulse solver with rigid-body dynamics; widely used for its accessibility and open-source nature.

DART / RaiSim. Alternative constraint formulations and integration schemes; compliant contact models.

Systematic evaluation across these platforms identifies behaviors that generalize (e.g., basic gait patterns, foot placement strategies) versus those that are simulator-dependent (e.g., exploitation of specific contact timing, resonance with numerical integration artifacts).

Mechanism Details: Different simulators solve the equations of motion differently. **MuJoCo** uses a soft-constraint model (via convex optimization), allowing small penetrations. **PyBullet** and **Isaac Gym** often use hard constraints or penalty-based methods depending on configuration.

A policy that cheats by exploiting MuJoCo’s soft contacts to generate extra ground force will fail in Isaac Gym or on a real robot.

Locomotion Successes:

- **Benchmark (Collins et al. [9]):** showed that policies validated on both Isaac Gym and MuJoCo had a $>90\%$ success rate on the real robot, whereas policies that worked only in one simulator had $<50\%$ transfer success.

Failure Modes:

- **Simulator Artifact Overfitting:** In PyBullet, high-frequency toe flapping can sometimes generate phantom forces due to integration errors, allowing a robot to move without proper traction. This behavior fails instantly on hardware. Sim-to-sim validation catches this by showing the policy failing in a simulator with a different integrator [9].

Conclusion: Sim-to-sim validation is a robust filter for simulation hacks. It forces the policy to learn physics-agnostic behaviors rather than integrator-dependent tricks.

3.10 Physics Engine Selection and Contact Modeling

Choosing the appropriate contact formulation and physics engine is a proactive strategy to minimize the sim-to-real gap, particularly for dynamic locomotion where impact mechanics are critical. Different solvers offer distinct trade-offs between physical accuracy, numerical stability, and computational speed.

3.10.1 Solver Methodologies

The mathematical core of contact simulation is the resolve of non-penetration and friction constraints. For a system with generalized coordinates $\mathbf{q}$ and velocities $\mathbf{v}$, the contact forces λ are often solved via a Linear Complementarity Problem (LCP):

$$0 \leq \lambda \perp \mathbf{M}\lambda + \mathbf{q} \geq 0 \quad (15)$$

where $\perp$ denotes orthogonality (i.e., $\lambda^T(\mathbf{M}\lambda + \mathbf{q}) = 0$). This ensures that contact forces are only active when bodies are in contact and that they are always repulsive.

Penalty Methods: Common in GPU-accelerated environments like Isaac Gym, these methods model contact forces as a spring-damper function: $F_c = -k\Delta x - d\dot{x}$. They are highly parallelizable but require careful tuning; overly soft contacts can cause feet to sink unphysically into the ground, while overly stiff contacts can lead to numerical explosions during high-speed impacts.

Constraint-Based Solvers: Used in MuJoCo [44] via projected Gauss-Seidel or similar iterative methods. These solve for contact forces that strictly prevent penetration (up to a tolerance). They generally offer better physical fidelity for rigid contacts but can struggle with the complex, over-constrained contact configurations often found in multi-legged robots on rough terrain.

Time-Stepping Schemes: Approaches like the Stewart-Trinkle formulation treat contact and dynamics as a unified problem, guaranteeing solution existence. They are robust but computationally expensive, making them less suitable for massive-scale RL training.

Application to Locomotion: Selecting a solver that matches the target terrain compliance is crucial. For instance, running on soft soil might be better approximated by penalty methods with tuned stiffness, whereas walking on concrete requires the rigidity of constraint-based solvers to prevent unphysical energy sinks. A mismatch often leads to moon-walking policies that slide effortlessly in simulation but fail to generate traction on real ground.

Locomotion Successes:

- **MuJoCo Benchmark (Todorov et al. [44]):** The constraint-based solver in MuJoCo has become the de facto standard for rigid-body locomotion research, enabling high-fidelity transfer for robots like the Unitree A1 and ANYmal on hard surfaces.

Failure Modes:

- **Contact Instability:** Using penalty methods with insufficient solver iterations or poorly tuned stiffness can result in feet penetrating the floor or vibrating violently, preventing the learning of stable gait cycles [9].

Conclusion: The physics engine is not just a computational backend but a core modeling choice. Matching the solver’s physical assumptions to the robot’s operating domain is a prerequisite for successful transfer.

4 Strengths, Limitations, and Open Challenges

4.1 Current Capabilities

Learning-based locomotion has achieved remarkable capabilities that substantially exceed those of classical model-based controllers in several dimensions:

Dynamic Agility: Quadrupedal robots executing running gaits exceeding 5 m/s, dynamic jumping maneuvers with flight phases, and recovery from significant external perturbations [4, 18].

Terrain Generalization: Blind proprioceptive policies successfully navigating stairs, slopes, and uneven natural terrain [23]; perceptive vision-based policies handling extreme terrain including rocky surfaces and vegetation [28].

Training Efficiency: Complete locomotion policy training accomplished in 20–60 minutes of wall-clock time using massively parallel GPU simulation [35, 39], representing orders-of-magnitude improvement over prior methods.

Embodiment Generality: Successful transfer of learned locomotion principles to humanoid robots, bipedal platforms, and hexapod configurations with minimal architectural modification [34, 37].

4.2 Persistent Limitations

Despite substantial progress, several fundamental limitations continue to constrain practical deployment:

- (1) **Absence of Formal Guarantees:** DRL policies are neural network function approximators without provable stability margins, Lyapunov certificates, or worst-case performance bounds. While empirical robustness may be substantial, principled safety assurance remains elusive, limiting deployment in safety-critical applications [5, 42].

- (2) **Reward Engineering Fragility:** Policy quality depends critically on careful reward function design. Minor reward modifications frequently produce dramatically different—and often unintended—behaviors, while reward hacking (achieving high reward through exploitation of simulator artifacts) remains prevalent. This labor-intensive, iterative process requires substantial domain expertise and empirical tuning.
- (3) **Energy Inefficiency:** Policies optimized primarily for task success and robustness often exhibit energetically inefficient motor patterns: excessive forces, agonist-antagonist co-contraction, high-frequency oscillations, and suboptimal gait timing. While energy-aware formulations exist [3, 13], efficiency typically remains secondary to robustness in objective hierarchies.
- (4) **Hardware Degradation:** Most approaches implicitly assume time-invariant robot dynamics, neglecting wear, thermal effects, lubrication changes, and component degradation that accumulate over extended operation. Policies may gradually degrade as hardware characteristics drift from training-time conditions, necessitating periodic recalibration or retraining.
- (5) **Out-of-Distribution Brittleness:** While learned policies exhibit impressive generalization within training distributions, out-of-distribution scenarios—novel terrain types, extreme payloads, unanticipated obstacles, or adversarial conditions—can trigger sudden, catastrophic failure modes with minimal warning.

4.3 Open Research Challenges

Several fundamental challenges define the research frontier and require continued investigation:

Safety-Constrained Learning: Developing principled methods to integrate hard safety constraints, such as joint position and velocity limits, contact force bounds, collision avoidance, and energy limits, directly into learning objectives while preserving task performance. Control barrier functions, constrained policy optimization, and safe exploration methods represent promising but incompletely explored directions.

Cross-Morphology Transfer: Creating techniques that transfer locomotion skills, representations, or learning curricula across robot morphologies with different kinematic structures, actuation characteristics, and mass distributions. Morphology-agnostic policy representations could dramatically reduce training requirements for novel platforms.

Multi-Modal Perception Fusion: Combining visual, tactile, proprioceptive, and potentially auditory modalities for robust perception that degrades gracefully under individual sensor failures. Current approaches typically privilege single modalities; principled fusion remains an open problem.

Lifelong Learning and Continual Adaptation: Enabling continuous policy adaptation to changing hardware conditions and environmental statistics without catastrophic forgetting of previously acquired skills. This requires balancing plasticity (adaptation to new conditions) with stability (retention of prior competence).

Interpretable and Verifiable Policies: Developing neural network architectures, training objectives, and post-hoc analysis techniques that provide insight into policy decision-making. Interpretability supports debugging, formal verification, trust calibration, and regulatory compliance for deployment in human environments.

5 Toward a Structured Sim-to-Real Pipeline

Synthesizing insights from the reviewed literature, I propose a multi-stage pipeline that systematically addresses the sim-to-real gap while leveraging complementary transfer methodologies. This Sim-to-Sim $\rightarrow$ Sim-to-Real pipeline provides a structured framework for practitioners.

5.1 Stage 1: Physics-Grounded Simulation Development

The pipeline commences with developing a high-fidelity simulation environment grounded in empirical physical measurements:

- (a) **Actuator Characterization:** Systematically measure motor electrical parameters, friction characteristics across the operating envelope, and bandwidth limitations. Train learned actuator networks or calibrate physics-based motor models from hardware telemetry [17].
- (b) **Mass and Inertia Identification:** Begin with CAD-derived estimates refined through pendulum oscillation and torsional rigidity measurements to obtain accurate link dynamics parameters.
- (c) **Sensor Modeling:** Characterize encoder resolution and noise spectra, IMU bias stability and noise power spectral densities, and communication latency distributions. Implement realistic sensor models incorporating delays, quantization, and stochastic noise.

This stage establishes the *nominal* simulation configuration that forms the distribution center for subsequent randomization protocols.

5.2 Stage 2: Latency-Aware Policy Training

Training incorporates temporal realism through explicit modeling of system delays and asynchrony:

- (a) **Observation Delay:** Inject randomized observation delays sampled from characterized distributions: $\tau_{\text{obs}} \sim \mathcal{U}[\tau_{\text{obs}}^{\min}, \tau_{\text{obs}}^{\max}]$.
- (b) **Action Delay:** Buffer commanded actions by estimated actuation latency before application to simulated dynamics.
- (c) **Jitter Injection:** Randomize simulation timestep duration around nominal values to emulate timing variability.
- (d) **Observation History:** Provide policies with observation history windows to enable implicit velocity estimation and predictive state inference despite delays.

5.3 Stage 3: Structured Domain Randomization

Apply domain randomization with physically-motivated structure and principled curriculum:

- (a) **Categorized Parameters:** Independently randomize dynamics parameters (masses, friction), actuator parameters (gains, limits, friction), and external disturbances (push forces, terrain variations) with separate distributions reflecting their distinct uncertainty sources.
- (b) **Curriculum Scheduling:** Initialize with narrow randomization ranges centered on identified parameters; progressively expand as policy performance exceeds threshold criteria.
- (c) **Privileged Information:** Train teacher policies with access to ground-truth extrinsic vectors encoding environment parameters, enabling subsequent adaptation module development.

5.4 Stage 4: Sim-to-Sim Validation

Prior to hardware deployment, validate policy robustness through cross-simulator evaluation:

- (a) **Multi-Platform Evaluation:** Systematically test trained policies in MuJoCo, Isaac Gym, PyBullet, and other available simulation environments.
- (b) **Failure Mode Analysis:** Identify behaviors that differ qualitatively across simulators; these discrepancies likely indicate exploitation of simulator-specific artifacts.
- (c) **Iterative Refinement:** If sim-to-sim transfer fails, return to Stage 3 to broaden randomization or Stage 1 to improve base simulation fidelity before proceeding.

This validation stage provides a strong predictive signal for real-world transfer at minimal hardware cost or risk.

5.5 Stage 5: Hierarchical Deployment with Online Adaptation

Deployment integrates learned policies within hierarchical control architectures with adaptation capabilities:

- (a) **Hierarchical Control Integration:** Embed the learned policy within a control hierarchy incorporating classical safety monitors, emergency stop logic, and fallback controllers that intervene when policy outputs exceed safety bounds.
- (b) **Adaptation Module Deployment:** Deploy trained environment encoders to enable real-time inference of terrain properties, friction estimates, and payload conditions via RMA or equivalent frameworks [22].
- (c) **Operational Data Collection:** Collect telemetry during deployment for potential offline analysis, system re-identification, and future policy refinement iterations.

5.6 Pipeline Summary

Table 2 summarizes the proposed pipeline stages, objectives, and principal techniques.

Table 2: Structured Sim-to-Sim $\rightarrow$ Sim-to-Real Pipeline

Stage	Name	Objective	Key Techniques
1	Simulation Development	Physical fidelity	System ID, actuator modeling
2	Latency-Aware Training	Temporal realism	Delay injection, history
3	Domain Randomization	Robustness	Structured DR, curriculum
4	Sim-to-Sim Validation	Artifact detection	Multi-simulator testing
5	Deployment	Real-world transfer	Adaptation, safety hierarchy

This structured approach ensures systematic coverage of sim-to-real gap sources while providing early validation before expensive hardware testing.

6 Conclusion

The sim-to-real transfer of learned locomotion policies remains a central, defining challenge in legged robotics. The reality gap arises from a complex interplay of modeling deficiencies—including actuator nonlinearities, contact mechanics approximations, and unmodeled thermal

and wear effects—sensing imperfections, and temporal constraints that collectively distinguish idealized simulation from physical reality.

This review has systematically examined the primary sources of this gap and comprehensively surveyed the methodological landscape developed to address them. Domain randomization provides baseline distributional robustness but risks producing overly conservative behaviors and cannot address dynamics outside its support. System identification aligns simulation with measured reality but requires careful experimental protocols and may struggle with rapidly time-varying parameters. Residual learning leverages classical control priors while enabling learned compensation for unmodeled effects. Online adaptation with latent representations enables real-time behavioral adjustment to changing conditions. Sim-to-sim validation provides an efficient, low-risk predictive filter for real-world transfer success.

State-of-the-art deployment systems increasingly integrate multiple complementary techniques within structured pipelines that address different gap sources at appropriate development stages. The proposed Sim-to-Sim $\rightarrow$ Sim-to-Real pipeline synthesizes these approaches, emphasizing physics-grounded simulation development, systematic cross-platform validation, and hierarchical deployment with online adaptation.

Looking forward, continued progress will require deeper physical alignment between simulation and hardware through more accurate actuator, contact, and deformation models; advances in safety-constrained and verifiable learning; techniques for cross-morphology skill transfer; principled multi-modal perception fusion; and interpretable policy architectures supporting trust calibration and regulatory compliance. Addressing these challenges is essential for realizing the vision of legged robots that can operate safely, efficiently, and autonomously across the diversity of real-world environments.

As the field matures, I anticipate convergence toward standardized evaluation benchmarks, validated simulation platforms, and community best practices that will accelerate reliable deployment of learned locomotion across diverse robotic platforms.

References

- [1] Ananye Agarwal et al. “Legged Locomotion in Challenging Terrains Using Egocentric Vision”. In: *Conference on Robot Learning* (2023).
- [2] Mihai Anitescu. “Optimization-Based Simulation of Nonsmooth Rigid Multibody Dynamics”. In: *Mathematical Programming* 105.1 (2006), pp. 113–143.
- [3] Michel Aractingi et al. “Controlling the Solo12 Quadruped Robot with Deep Reinforcement Learning”. In: *Scientific Reports* 13.1 (2023), pp. 1–13.
- [4] Guillaume Bellegarda and Quan Nguyen. “Robust High-Speed Running for Quadruped Robots Via Deep Reinforcement Learning”. In: *IEEE Robotics and Automation Letters* 7.3 (2022), pp. 8043–8050.
- [5] Felix Berkenkamp et al. “Safe Model-Based Reinforcement Learning with Stability Guarantees”. In: *Advances in Neural Information Processing Systems* 30 (2017).
- [6] Steven Bohez et al. “Imitate and Repurpose: Learning Reusable Robot Movement Skills From Human and Animal Behaviors”. In: *arXiv preprint arXiv:2203.17138* (2022).
- [7] Yevgen Chebotar et al. “Closing the Sim-to-Real Loop: Adapting Simulation Randomization with Real World Experience”. In: *IEEE International Conference on Robotics and Automation*. IEEE. 2019, pp. 8973–8979.

- [8] Dian Chen et al. “Learning by Cheating”. In: *Conference on Robot Learning* (2020), pp. 66–75.
- [9] Jack Collins, Ross Brown, and Jürgen Leitner. “A Review of Physics Simulators for Robotic Applications”. In: *IEEE Access* 9 (2021), pp. 51416–51431.
- [10] Erwin Coumans and Yunfei Bai. “PyBullet, A Python Module for Physics Simulation for Games, Robotics and Machine Learning”. In: (2021).
- [11] Siamak Fakoorian, Shounak Chatterjee, and Dan Simon. “Meta-Learning for Control of Legged Locomotion”. In: *IEEE Access* 10 (2022), pp. 55555–55567.
- [12] Chelsea Finn, Pieter Abbeel, and Sergey Levine. “Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks”. In: *International Conference on Machine Learning*. PMLR, 2017, pp. 1126–1135.
- [13] Zipeng Fu et al. “Minimizing Energy Consumption Leads to the Emergence of Gaits in Legged Robots”. In: *Conference on Robot Learning* (2021), pp. 928–937.
- [14] Siddhant Gangapurwala et al. “RLOC: Terrain-Aware Legged Locomotion Using Reinforcement Learning and Optimal Control”. In: *IEEE Transactions on Robotics* 38.5 (2022), pp. 2908–2927.
- [15] Yaroslav Ganin et al. “Domain-Adversarial Training of Neural Networks”. In: *Journal of Machine Learning Research* 17.59 (2016), pp. 1–35.
- [16] Tuomas Haarnoja et al. “Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor”. In: *International Conference on Machine Learning* (2018), pp. 1861–1870.
- [17] Jemin Hwangbo, Joonho Lee, and Marco Hutter. “Learning Actuator Dynamics from Time-Series Sensor Data”. In: *Science Robotics* 4.26 (2019).
- [18] Jemin Hwangbo et al. “Learning Agile and Dynamic Motor Skills for Legged Robots”. In: *Science Robotics* 4.26 (2019), eaau5872.
- [19] Atil Iscen et al. “Policies Modulating Trajectory Generators”. In: *Conference on Robot Learning* (2018), pp. 916–926.
- [20] Gwanghyeon Ji et al. “Concurrent Training of a Control Policy and a State Estimator for Dynamic and Robust Legged Locomotion”. In: *IEEE Robotics and Automation Letters* 7.2 (2022), pp. 4630–4637.
- [21] Tobias Johannink et al. “Residual Policy Learning for Shared Control”. In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2019, pp. 6023–6029.
- [22] Ashish Kumar et al. “RMA: Rapid Motor Adaptation for Legged Robots”. In: *Robotics: Science and Systems*. 2021.
- [23] Joonho Lee et al. “Learning Quadrupedal Locomotion over Challenging Terrain”. In: *Science Robotics* 5.47 (2020), eabc5986.
- [24] Zhongyu Li et al. “Reinforcement Learning for Robust Parameterized Locomotion Control of Bipedal Robots”. In: *IEEE International Conference on Robotics and Automation* (2021), pp. 2811–2817.
- [25] Zhongyu Li et al. “Reinforcement Learning for Versatile, Dynamic, and Robust Bipedal Locomotion Control”. In: *International Journal of Robotics Research* (2024).
- [26] Viktor Makoviychuk et al. “Isaac Gym: High Performance GPU-Based Physics Simulation for Robot Learning”. In: *Advances in Neural Information Processing Systems* (2021).

- [27] Bhairav Mehta et al. “Active Domain Randomization”. In: *Conference on Robot Learning* (2020), pp. 1162–1176.
- [28] Takahiro Miki et al. “Learning Robust Perceptive Locomotion for Quadrupedal Robots in the Wild”. In: *Science Robotics* 7.62 (2022), eabk2822.
- [29] Fabio Muratore et al. “Assessing Transferability from Simulation to Reality for Reinforcement Learning”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence*. 2018.
- [30] Fabio Muratore et al. “Robot Learning from Randomized Simulations: A Review”. In: *Frontiers in Robotics and AI* 9 (2022), p. 799893.
- [31] Xue Bin Peng et al. “DeepMimic: Example-Guided Deep Reinforcement Learning of Physics-Based Character Skills”. In: *ACM Transactions on Graphics (TOG)*. Vol. 37. 4. 2018, pp. 1–14.
- [32] Xue Bin Peng et al. “Sim-to-Real Transfer of Robotic Control with Dynamics Randomization”. In: *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE. 2018, pp. 1–8.
- [33] Lerrel Pinto et al. “Robust Adversarial Reinforcement Learning”. In: *International Conference on Machine Learning*. PMLR. 2017, pp. 2817–2826.
- [34] Ilija Radosavovic et al. “Real-World Humanoid Locomotion with Reinforcement Learning”. In: *Science Robotics* 9.89 (2024), eadi9579.
- [35] Nikita Rudin et al. “Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning”. In: *Conference on Robot Learning* (2022), pp. 91–100.
- [36] John Schulman et al. “Proximal Policy Optimization Algorithms”. In: *arXiv preprint arXiv:1707.06347* (2017).
- [37] Jonah Siekmann et al. “Blind Bipedal Stair Traversal via Sim-to-Real Reinforcement Learning”. In: *Robotics: Science and Systems* (2021).
- [38] Tom Silver et al. “Residual Policy Learning”. In: *Advances in Neural Information Processing Systems*. 2018.
- [39] Laura Smith, Ilya Kostrikov, and Sergey Levine. “Walk in the Park: Learning to Walk in 20 Minutes With Model-Free Reinforcement Learning”. In: *arXiv preprint arXiv:2208.07860* (2022).
- [40] David E Stewart. “Rigid-Body Dynamics with Friction and Impact”. In: *SIAM Review* 42.1 (2000), pp. 3–39.
- [41] Jie Tan et al. “Sim-to-Real: Learning Agile Locomotion for Quadruped Robots”. In: *Robotics: Science and Systems*. 2018.
- [42] Brijen Thananjeyan et al. “Recovery RL: Safe Reinforcement Learning with Learned Recovery Zones”. In: *IEEE Robotics and Automation Letters* 6.3 (2021), pp. 4915–4922.
- [43] Josh Tobin et al. “Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World”. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE. 2017, pp. 23–30.
- [44] Emanuel Todorov, Tom Erez, and Yuval Tassa. “MuJoCo: A Physics Engine for Model-Based Control”. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE. 2012, pp. 5026–5033.

- [45] Ruihan Yang et al. “Learning Vision-Guided Quadrupedal Locomotion End-to-End with Cross-Modal Transformers”. In: *International Conference on Learning Representations* (2022).
- [46] Wenhao Yu et al. “Preparing for the Unknown: Learning a Universal Policy with Online System Identification”. In: *Robotics: Science and Systems* (2017).